Correção do exercício 6

Três conferências para cada consulta

1º passo

A consulta de dentro

Ela roda sozinha e devolve a lista ou o valor que você esperava?

2º passo

O número de linhas

O resultado tem as linhas que o «Confira» do exercício pede?

3º passo

A view

A view mostra o mesmo resultado da consulta?

Window Functions e View Materializada

Banco de Dados 2 · Aula 15

SQL: Structured Query Language · CTE: Common Table Expression

Antes de começar: chute uma resposta

OBJETIVO **Qual é o produto mais vendido de cada categoria?**

1 · Pense

Como você faria?

Escreva como você faria com o GROUP BY que já conhece.

2 · Compare

Onde o GROUP BY para

A aula vai mostrar onde o GROUP BY para e o que vem depois dele.

Uma pergunta da aula passada, de memória: a consulta de dentro devolve uma lista ou um valor?

NOVO ASSUNTO

Window functions

Uma window function calcula um número para cada linha olhando as linhas vizinhas, sem juntar as linhas em uma só. O GROUP BY resume o grupo em uma linha:

- A window function mantém todas as linhas e acrescenta uma coluna.
- Essa coluna pode ser a posição no ranking ou a soma até ali.

Assim cada linha continua visível, com o número do grupo ao lado.

Onde e por que usar window functions

Quatro perguntas da loja que pedem uma posição ou uma soma linha a linha

Posição

Ranking por grupo

Qual é o produto mais vendido de cada categoria? A posição recomeça em cada categoria.

Posição

Os melhores de cada lugar

Quem são os clientes que mais gastaram em cada estado? O ranking recomeça em cada UF (Unidade Federativa).

Acumulado

Soma até aqui

Quanto a loja já faturou do primeiro mês até este? A soma cresce mês a mês.

Contexto

Comparar com o grupo

Este produto vendeu muito ou pouco perto dos outros da categoria? A linha continua visível, com o número do grupo ao lado.

Uma pergunta que o GROUP BY não responde

OBJETIVO **Qual é o produto mais vendido de cada categoria?**

GROUP BY

Resume

Soma as unidades de cada produto. Mas não diz quem ficou em 1º lugar dentro da categoria.

RANK

Numera

Dá a posição de cada produto dentro da categoria dele. Com a posição, basta pegar o 1º.

A palavra OVER é o que transforma uma função em window function: ela diz em que grupo e em que ordem contar.

A consulta pronta (1 de 2)

OBJETIVO **Qual é o produto mais vendido de cada categoria?**

```sql
WITH ranking AS (   -- etapa: a posição de cada produto
  SELECT cat.nome AS categoria, pr.nome AS produto,   -- nomes
    RANK() OVER (PARTITION BY cat.nome   -- conta por categoria
      ORDER BY SUM(i.quantidade) DESC) AS posicao   -- mais = 1
  FROM item_pedido i   -- itens de pedido (apelido i)
  JOIN pedido p ON p.id = i.pedido_id   -- liga ao pedido
  JOIN produto pr ON pr.id = i.produto_id   -- ao produto
  JOIN categoria cat ON cat.id = pr.categoria_id   -- à categoria
```

A etapa dá a cada produto a sua posição dentro da categoria. Falta ficar só com o 1º lugar.

A consulta pronta (2 de 2)

OBJETIVO **Qual é o produto mais vendido de cada categoria?**

```
1 ...   -- a etapa do slide anterior
2   WHERE p.status IN ('pago', 'enviado', 'entregue')   --
      vendas
3   GROUP BY cat.nome, pr.nome)   -- um grupo por produto
4 SELECT categoria, produto   -- o que mostrar
5 FROM ranking   -- a etapa de cima
6 WHERE posicao = 1   -- só o 1º de cada categoria
7 ORDER BY categoria;   -- em ordem alfabética
```

Resultado: 7 linhas para 5 categorias. Em Armazenamento, três produtos venderam o mesmo tanto e dividem o 1º lugar. Vamos montar em três passos.

Passo 1: quanto vendeu cada produto

OBJETIVO **Primeiro, as unidades vendidas de cada produto (1 de 3)**

```sql
SELECT cat.nome AS categoria, pr.nome AS produto,   -- nomes
  SUM(i.quantidade) AS unidades   -- soma das unidades
FROM item_pedido i   -- itens de pedido (apelido i)
JOIN pedido p ON p.id = i.pedido_id   -- liga ao pedido
JOIN produto pr ON pr.id = i.produto_id   -- ao produto
JOIN categoria cat ON cat.id = pr.categoria_id   -- à categoria
WHERE p.status IN ('pago', 'enviado', 'entregue')   -- vendas
GROUP BY cat.nome, pr.nome;   -- um grupo por produto
```

Antes de rodar, preveja: quantas linhas? Resultado: 11 linhas, uma para cada produto vendido. Ainda não há posição.

Passo 2: a posição dentro da categoria

OBJETIVO **Agora, a posição de cada produto na categoria dele (2 de 3)**

```sql
SELECT cat.nome AS categoria, pr.nome AS produto,  -- nomes
  SUM(i.quantidade) AS unidades,  -- soma das unidades
  RANK() OVER (PARTITION BY cat.nome  -- recomeça por categoria
    ORDER BY SUM(i.quantidade) DESC) AS posicao  -- mais = 1
FROM item_pedido i  -- o mesmo FROM do passo 1
...  -- os mesmos JOIN, WHERE e GROUP BY do passo 1
```

Continuam as 11 linhas: o RANK não junta linhas, só acrescenta a coluna posicao. Empate recebe a mesma posição.

Dentro do OVER

As duas partes dizem onde contar e em que ordem (3 de 3)

Onde

PARTITION BY

Separa as linhas em grupos. A contagem recomeça em cada grupo, aqui em cada categoria.

Ordem

ORDER BY

Diz quem vem primeiro dentro do grupo. Com DESC, quem vendeu mais fica com a posição 1.

Sem PARTITION BY, o ranking é um só, para a loja inteira.

Passo 3: ficar só com o 1º lugar

OBJETIVO **Por fim, só o produto mais vendido de cada categoria**

```sql
WITH ranking AS (   -- etapa: o passo 2 ganha um nome
   ...   -- aqui entra a consulta do passo 2
)   -- fecha a etapa
SELECT categoria, produto   -- o que mostrar
FROM ranking   -- usa a etapa como se fosse tabela
WHERE posicao = 1   -- só o 1º de cada categoria
ORDER BY categoria;   -- em ordem alfabética
```

O WHERE não enxerga a posição na mesma consulta, porque o RANK é calculado depois dele. Por isso o passo 2 vira uma etapa da CTE (Common Table Expression) e o filtro fica na consulta de fora.

GROUP BY ou OVER?

Os dois trabalham com grupos, mas devolvem coisas diferentes

GROUP BY

Uma linha por grupo

Junta as linhas. Bom para «quanto vendeu cada categoria».

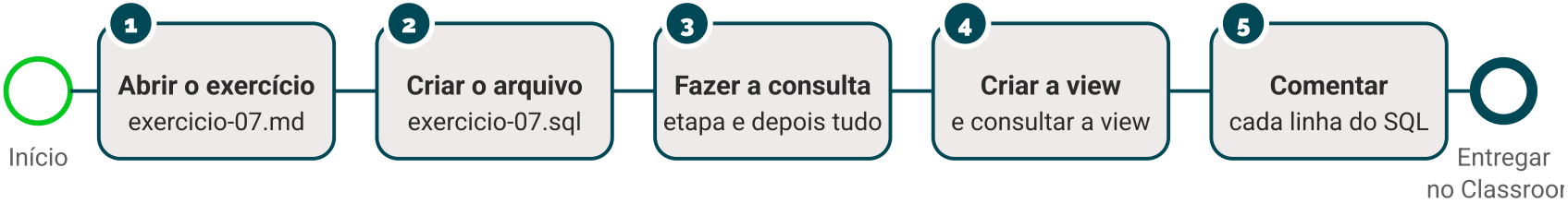
OVER

Todas as linhas, com um número a mais

Mantém as linhas. Bom para «qual a posição deste produto na categoria».

Exercício 7: window functions e view materializada

Pode fazer em dupla. Cada aluno entrega o seu, no Classroom



> Faça uma consulta de cada vez e repita os passos 3 a 5 até a consulta 10. Escreva um comentário (--) em cada linha do SQL.

Consulta 1: modelo resolvido (1 de 2)

OBJETIVO **Quem são os 3 clientes que mais gastaram em cada estado?**

```sql
WITH gasto AS (  -- etapa: gasto por cliente e UF
  SELECT p.dest_uf AS uf, c.nome AS cliente,  -- a UF e o nome
    SUM(p.total) AS gasto,  -- soma dos pedidos do cliente
    RANK() OVER (PARTITION BY p.dest_uf  -- recomeça por UF
      ORDER BY SUM(p.total) DESC) AS posicao  -- mais = 1
  FROM pedido p  -- pedidos (apelido p)
  JOIN cliente c ON c.id = p.cliente_id  -- liga ao cliente
```

Digite, rode a etapa sozinha e reescreva os comentários com as suas palavras. UF: Unidade Federativa.

Consulta 1: modelo resolvido (2 de 2)

OBJETIVO **Quem são os 3 clientes que mais gastaram em cada estado?**

```sql
...   -- a etapa do slide anterior
  WHERE p.status IN ('pago', 'enviado', 'entregue')   -- vendas
  GROUP BY p.dest_uf, c.nome)   -- um grupo por cliente e UF
SELECT uf, cliente, gasto, posicao   -- o que mostrar
FROM gasto   -- a etapa de cima
WHERE posicao ⩽ 3   -- os 3 primeiros de cada UF
ORDER BY uf, posicao;   -- por UF e pela posição
```

Resultado: 25 linhas para 8 estados. Há empates: em MG, três clientes dividem o 1º lugar, e em RS quatro clientes entram no top 3.

A view da consulta 1

OBJETIVO **Guardar a consulta 1 com um nome**

```sql
CREATE OR REPLACE VIEW vw_top3_clientes_por_uf AS   -- o nome
...   -- aqui entra a consulta 1 inteira, do WITH ao ORDER BY
;   -- fim da view
SELECT * FROM vw_top3_clientes_por_uf;   -- usa como tabela
```

A view mostra as mesmas 25 linhas da consulta. Agora faça as consultas 1 a 4 do exercício.

DEMONSTRAÇÃO

EXPLAIN

O banco escolhe sozinho como buscar os dados. O EXPLAIN mostra essa escolha, o plano de execução:

- Hoje é só para ver: você assiste e prevê o que o banco vai fazer.
- Não há exercício e o assunto não cai na prova.

Vale guardar a ideia, porque ela explica por que uma consulta pode ser lenta.

O que o EXPLAIN mostra

OBJETIVO **Como o banco acha os pedidos de um cliente?**

```
1 EXPLAIN ANALYZE   -- mostra o plano e roda a consulta de verdade
2 SELECT * FROM pedido   -- todos os dados do pedido
3 WHERE cliente_id = 7;   -- só os pedidos do cliente 7
```

Antes de rodar, preveja: o banco vai ler a tabela inteira ou vai direto aos pedidos do cliente 7?

O que olhar no plano

Três coisas, e só elas

Varredura

Seq Scan ou Index Scan

Seq Scan lê a tabela toda, linha por linha. Index Scan usa um índice e vai direto às linhas.

Custo

cost

A estimativa do banco para o trabalho. Compare o antes e o depois.

Tempo

Execution Time

Quanto a consulta levou de verdade, em milissegundos.

Depois do índice

OBJETIVO **O índice muda o plano?**

```sql
CREATE INDEX idx_pedido_cliente  -- cria um índice com este nome
ON pedido (cliente_id);  -- na coluna usada no WHERE
EXPLAIN ANALYZE  -- o mesmo plano de antes
SELECT * FROM pedido WHERE cliente_id = 7;  -- a mesma busca
```

Com poucos pedidos, o banco continua lendo a tabela toda: ela é pequena e o índice não compensa. Com muitos pedidos, a história muda.

E com muitos pedidos?

OBJETIVO **O índice compensa numa tabela grande?**

```
1 EXPLAIN ANALYZE   -- o plano, agora numa cópia grande
2 SELECT * FROM pedido_volume   -- 3 milhões de pedidos de teste
3 WHERE cliente_id = 7;   -- só os pedidos do cliente 7
4 CREATE INDEX idx_volume_cliente   -- cria o índice
5 ON pedido_volume (cliente_id);   -- e roda o plano de novo
```

Antes do índice: Seq Scan nos 3 milhões de linhas. Depois: Bitmap Index Scan, com custo e tempo bem menores (no teste, de cerca de 100 ms para menos de 1 ms).

NOVO ASSUNTO

Soma acumulada

A soma acumulada responde «quanto já foi até aqui». Cada linha mostra o valor dela e, ao lado, a soma de todas as linhas anteriores mais ela:

- É a mesma ideia do OVER que você acabou de ver.
- Agora com o SUM no lugar do RANK.

A soma cresce linha a linha, e a última linha mostra o total.

A consulta pronta (1 de 2)

OBJETIVO **Quanto a loja faturou em cada mês, e quanto já somou até ali?**

```sql
WITH por_mes AS (   -- etapa: o faturamento de cada mês
  SELECT DATE_TRUNC('month', criado_em)::date AS mes,   -- o mês
    SUM(total) AS faturamento   -- soma dos pedidos do mês
  FROM pedido   -- a tabela dos pedidos
  WHERE status IN ('pago', 'enviado', 'entregue')   -- só vendas
  GROUP BY 1)   -- um grupo por mês (a 1ª coluna)
```

Rode a etapa sozinha primeiro: ela dá uma linha por mês, com o faturamento só daquele mês.

A consulta pronta (2 de 2)

OBJETIVO **Quanto a loja faturou em cada mês, e quanto já somou até ali?**

```sql
...   -- a etapa do slide anterior
SELECT mes, faturamento,   -- o mês e o valor dele
  SUM(faturamento) OVER (ORDER BY mes) AS acumulado   -- até aqui
FROM por_mes   -- a etapa de cima
ORDER BY mes;   -- do mês mais antigo ao mais novo
```

Resultado: uma linha por mês. O acumulado da última linha é o total de todas as vendas.

Como a soma cresce

Rode a etapa sozinha e depois a consulta inteira

1 · por_mes

A etapa

Uma linha por mês, com o faturamento só daquele mês.

2 · acumulado

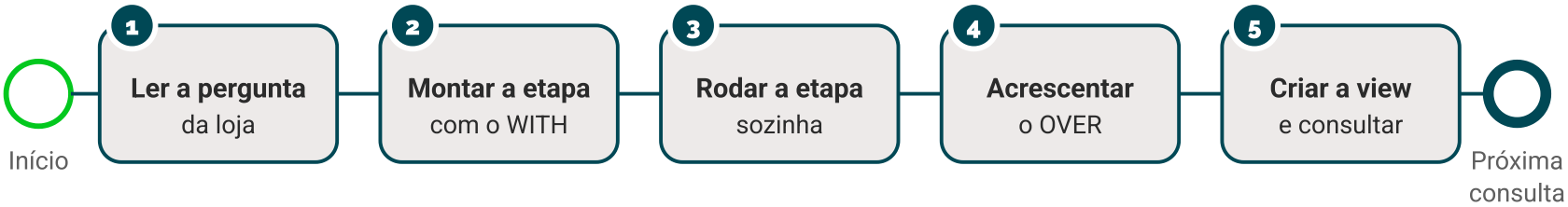
O OVER

Em cada linha, soma o mês dela com todos os meses antes dele, na ordem do ORDER BY.

Sem o ORDER BY dentro do OVER, o banco soma tudo de uma vez, e o acumulado vira o total em todas as linhas.

Prática: consultas 5 a 7

Soma acumulada misturada com o que você já sabe



Antes de rodar tudo, escreva em um comentário quantas linhas você espera.

NOVO ASSUNTO

View materializada

A view simples guarda só a consulta: ela roda toda vez que você usa a view. A view materializada guarda também o resultado:

- Fica mais rápida de ler, porque não refaz a conta.
- Só muda quando você manda atualizar.

Por isso ela serve para relatórios pesados, que podem esperar a próxima atualização.

View ou view materializada?

As duas têm nome e são usadas como tabela

View

Guarda a consulta

Roda a consulta toda vez. Mostra sempre o dado de agora.

Materializada

Guarda o resultado

Lê o resultado guardado, sem refazer a conta. Só muda com REFRESH.

Use a materializada quando a conta é pesada e o dado pode esperar a próxima atualização, como um relatório mensal.

A consulta pronta (1 de 2)

OBJETIVO Guardar o faturamento de cada mês e categoria

```sql
CREATE MATERIALIZED VIEW mv_faturamento_mensal AS  -- guarda
SELECT DATE_TRUNC('month', p.criado_em)::date AS mes,  -- o mês
  cat.nome AS categoria,  -- a categoria
  SUM(i.quantidade * i.preco_unitario) AS faturamento  -- valor
FROM item_pedido i  -- itens de pedido (apelido i)
JOIN pedido p ON p.id = i.pedido_id  -- liga ao pedido
JOIN produto pr ON pr.id = i.produto_id  -- liga ao produto
```

```
8 JOIN categoria cat ON cat.id = pr.categoria_id   -- e à
      categoria
```

> Use o prefixo mv_ no nome. Falta o filtro das vendas e o agrupamento.

A consulta pronta (2 de 2)

OBJETIVO **Guardar o faturamento de cada mês e categoria**

```
1 ...   -- a consulta do slide anterior
2 WHERE p.status IN ('pago', 'enviado', 'entregue')   -- vendas
3 GROUP BY 1, cat.nome;   -- um grupo por mês e categoria
```

Resultado: uma linha para cada mês e categoria com venda. O resultado fica guardado dentro da view.

Atualizar com REFRESH

OBJETIVO **Fazer a view materializada mostrar as vendas novas**

```
1 SELECT * FROM mv_faturamento_mensal;   -- o resultado guardado
2 REFRESH MATERIALIZED VIEW mv_faturamento_mensal;   -- refaz
      a conta
3 SELECT * FROM mv_faturamento_mensal;   -- agora com o dado novo
```

Um pedido pago depois da criação só aparece na materializada depois do REFRESH. Na view simples, ele aparece na hora.

Treine o sorteio

Em dupla, como vai ser na prova

1 · Sorteie

Um colega escolhe

Sem avisar, uma consulta já feita.

2 · Explique

O outro roda e explica

Roda a consulta e explica cada parte em voz alta.

Depois, troquem de lugar.

Para fixar: responda de memória

Sem olhar o material e sem nota. Escreva as respostas em comentários, no fim do arquivo

Pergunta 1

Junta linhas?

O OVER junta as linhas como o GROUP BY? O que ele faz?

Pergunta 2

PARTITION BY

Para que serve o PARTITION BY dentro do OVER?

Pergunta 3

Quando muda?

Quando a view materializada mostra um dado novo?